Programming and Systems Development

Mock C - Practical Assessment Style: Python (40 marks)

Time allowed for this mock: 1 hour (the same as the real mid-term test). Open book: you may use your printed notes and practice materials. No communication with other candidates. No Tkinter/GUI is needed.

You should be comfortable with pandas, matplotlib, SQLite and a basic PyTorch network before you start. This paper tests the **individual operations** in each area. **Do not try to memorise one complete solution:** practise each operation separately and make sure you understand what every line of code is doing. Each operation earns its own marks, so a program that does most things correctly still scores well.

Write ALL four sections in ONE Python file, in the order given, and make sure the program runs from top to bottom without errors. Ensure you add short comments saying which task each part answers.

The data

Two CSV files are provided; place both in the same folder as your program.

- `customers.csv`: `CustomerID, Name, Region, Age, LoyaltyYears`
- `orders.csv`: `OrderID, CustomerID, OrderDate, Category, Items, Amount`

The two files share the column `CustomerID`. `Amount` is the order value in pounds and `OrderDate` is written `YYYY-MM-DD`. Ensure you do not assume the files are clean.

Section 1: Pandas [12 marks]

- **1a** Load both CSV files into DataFrames. [2 marks]
- **1b** Check and print the shape of each DataFrame. [1 mark]
- **1c** Identify the duplicate rows in each DataFrame, print how many there are in each, and remove them. [3 marks]
- **1d** Using the cleaned DataFrames, merge the orders with the customer details using the common column `CustomerID`. Use an **inner join**. Print the shape of the merged DataFrame and print how many orders were left out because their customer is not in `customers.csv`. [3 marks]
- **1e** From the merged DataFrame, select only the columns `OrderID`, `Name`, `Region`, `Category` and `Amount`, and keep only the rows where the region is `North` **and** the amount is greater than 50. Print how many rows remain and show the first five. [2 marks]
- **1f** Save the merged, cleaned DataFrame (all columns, all merged rows from 1d, no row-number column) to a CSV file called `merged_orders.csv`. [1 mark]

Section 2: Matplotlib [8 marks]

Using the merged DataFrame, create the following four plots, each in its own figure. **Every plot needs a suitable title and labels on both axes** (1 mark for the correct plot, 1 mark for the title and labels).

- **2a** A **scatter plot** of the number of items (x) against the order amount (y). [2 marks]
- **2b** A **line plot** of the total order amount for each month, in date order. [2 marks]
- **2c** A **bar chart** of the total order amount for each category. [2 marks]
- **2d** A **box plot** showing the distribution of the order amount for each region (one box per region, labelled). [2 marks]

Section 3: SQLite [10 marks]

- **3a** Create (or connect to) an SQLite database file `ShopDB.db` and store the merged DataFrame from Section 1 in a table called `Orders`. Ensure that running your program a second time does not duplicate the rows. Print the number of rows stored. [3 marks]
- **3b** Write a query that selects `OrderID`, `Name`, `Category` and `Amount` for the orders from region `North` whose amount is greater than 50, with the biggest amount first. The region and the amount must be passed to the query as **parameters** (using `?`), not typed into the SQL text. Print how many rows were returned and the first three. [2 marks]
- **3c** The shop gives every `Books` order a voucher worth 2.00 pounds. Write an **UPDATE** query (again with `?` parameters) that subtracts 2.00 from the `Amount` of every `Books` order. Show the total amount of `Books` orders **before** and **after** the update, print how many rows were updated, and make sure the change is saved to the database. [3 marks]
- **3d** Retrieve all `Books` orders (`OrderID`, `Name`, `Category`, `Amount`) with the biggest amount first, and export the retrieved rows, with column headings, to a CSV file called `books_after_update.csv`. Print how many rows were exported. [2 marks]

Section 4: PyTorch [10 marks]

- **4a** Ensure you set the random seed to **42** *before* generating any data, so that your results are the same every time you run the program. Then generate synthetic data for regression: **N = 200** samples and **F = 3** features, where `X` is drawn from a standard normal distribution and the continuous target is `y = X @ w + b + noise`, with `w = [2.0, −1.0, 0.5]` (a column of shape 3 x 1), `b = 0.7` and noise drawn from a normal distribution with standard deviation **0.1**. [3 marks]
- **4b** Print the shapes of `X` and `y`. [1 mark]
- **4c** Define a small neural network as a class that extends `torch.nn.Module` with an input layer of 3 features, one hidden Linear layer of 8 neurons followed by ReLU, and an output Linear layer of 1 neuron (no activation, because the target is a continuous number). [2 marks]
- **4d** Create the model. Use the loss function `MSELoss` and the optimiser `SGD` with a learning rate of 0.05. [1 mark]
- **4e** Train the network for **100 epochs** using all the data in each epoch (a basic training loop). Record the training loss of every epoch and print the loss at epochs 1, 21, 41, 61, 81 and 100. [2 marks]
- **4f** Plot the recorded training loss against the epoch number, with a title and axis labels. The curve should decrease, showing the network is learning. [1 mark]

Example output layout (placeholders only)

```
customers shape: (0, 0)
orders shape:    (0, 0)
duplicate customer rows: 0
duplicate order rows:    0
merged (inner join) shape: (0, 0)
orders with no matching customer: 0
...
Rows stored in Orders: 0
North orders over 50 (biggest first): 0
rows updated: 0
Books total BEFORE: 0.00   AFTER: 0.00
exported 0 rows to books_after_update.csv

X shape: (0, 0)  y shape: (0, 0)
Epoch   1/100  loss = 0.0000
```

Submission

Name your file exactly as instructed, `your_name_python_task.py`, and upload that single file. Marks are awarded per operation for correctness, and also for clear names, helpful comments and tidy layout.

Section	Marks
1 Pandas	12
2 Matplotlib	8
3 SQLite	10
4 PyTorch	10
Total	**40**